Review: Learnings from the Prometheus and Grafana Lab

1. Going through this lab, I gained a much better understanding of how Prometheus and Grafana work together for system monitoring. Before this, I knew in theory that Prometheus collects metrics and Grafana visualizes them, but I had not set them up myself in a containerized environment. This hands-on experience helped solidify that knowledge.

2. One of the key learnings for me was about container networking in Docker Compose. Initially, I tried to connect Grafana to Prometheus using localhost:9090, but it kept failing. Through troubleshooting, I realized that in Docker Compose, services communicate with each other using their service names, not localhost. Once I changed the data source URL in Grafana to http://prometheus:9090, it worked perfectly. This was a good reminder of how Docker networking works internally.

3. Another important lesson was the importance of correct YAML formatting. Prometheus refused to start at first because of a small indentation issue in the prometheus.yml file. YAML is very sensitive to spaces and structure, and this experience made me more attentive to such details in configuration files.

4. Also, while setting up the Prometheus configuration, I learned about how Prometheus scrapes metrics and how the scrape_interval and targets are defined. I understood better how Prometheus can be set up to monitor itself as well as other services.

5. In Grafana, I learned how to add Prometheus as a data source and build basic dashboards. Writing PromQL queries to visualize metrics like CPU usage, memory, and disk I/O gave me a clearer picture of how flexible and powerful Grafana is for monitoring various aspects of a system.

6. Overall, this lab helped me connect theory to practice. I now feel more confident setting up basic monitoring systems using Docker, Prometheus, and Grafana. It also taught me the value of debugging and paying attention to configuration details, which are essential skills in any DevOps or cloud-based environment.